

Chapter 1: AWS Interview Questions (Q1–Q40)

Q1. What is AWS IAM?

Answer:

AWS IAM (Identity and Access Management) securely manages access to AWS resources. It provides Users, Groups, Roles, and Policies to control permissions. In my project, I use IAM Roles for EC2, EKS, and Jenkins instead of access keys, following the principle of least privilege to improve security.

Q2. What is a VPC?

Answer:

A VPC (Virtual Private Cloud) is a logically isolated network in AWS where cloud resources are deployed securely. It contains public and private subnets, route tables, Internet Gateway, NAT Gateway, Security Groups, and NACLs. In my project, we use a multi-AZ VPC architecture for high availability and secure communication.

Q3. Difference between Public and Private Subnets?

Answer:

A public subnet has a route to the Internet Gateway, allowing internet access for resources like ALBs or Bastion Hosts. A private subnet has no direct internet access and uses a NAT Gateway for outbound connectivity. In my project, EKS worker nodes and databases are deployed in private subnets for better security.

Q4. What is an Internet Gateway?

Answer:

An Internet Gateway (IGW) enables communication between a VPC and the internet. It is attached to the VPC, and public route tables point to it. Resources in public subnets use the IGW to send and receive internet traffic. In my project, the Application Load Balancer uses the Internet Gateway.

Q5. What is a NAT Gateway?

Answer:

A NAT Gateway allows instances in private subnets to access the internet without exposing them to inbound internet traffic. It is deployed in a public subnet and uses the Internet Gateway. In my project, EKS worker nodes use the NAT Gateway to pull Docker images from Amazon ECR and install updates.

Q6. Why is the NAT Gateway placed in a Public Subnet?

Answer:

The NAT Gateway is placed in a public subnet because it requires internet connectivity through the Internet Gateway. It forwards outbound requests from private instances to the internet and returns the responses. This allows private resources to download updates or access external services while remaining inaccessible from the internet.

Q7. Difference between Security Group and NACL?

Answer:

A Security Group is a stateful firewall attached to an EC2 instance or network interface and supports only allow rules. A NACL is a stateless firewall applied at the subnet level and supports both allow and deny rules. We use Security Groups for instance security and NACLs as an additional subnet-level security layer.

Q8. What is an EC2 Instance?

Answer:

Amazon EC2 is a virtual server that provides scalable compute capacity in AWS. It allows you to run applications with different CPU, memory, and storage configurations. In my project, EC2 instances are used as EKS worker nodes to host containerized applications.

Q9. What is an AMI?

Answer:

An Amazon Machine Image (AMI) is a preconfigured template used to launch EC2 instances. It contains the operating system, application software, and configuration settings. AMIs ensure consistent deployments and reduce server provisioning time across different environments.

Q10. What is Auto Scaling?

Answer:

Auto Scaling automatically increases or decreases the number of EC2 instances based on demand or CloudWatch metrics. It improves application availability and optimizes infrastructure costs. In my project, Auto Scaling helps maintain application performance during traffic spikes without manual intervention.

Q11. What is an Application Load Balancer (ALB)?

Answer:

An Application Load Balancer distributes HTTP and HTTPS traffic across multiple targets. It supports path-based routing, host-based routing, SSL termination, and health checks. In my project, the ALB routes external user requests to applications running on Amazon EKS.

Q12. Difference between ALB and NLB?

Answer:

ALB operates at Layer 7 and supports HTTP/HTTPS traffic, path-based routing, and SSL termination. NLB operates at Layer 4 and handles TCP/UDP traffic with high performance and low latency. We use ALB for web applications and NLB for TCP-based applications.

Q13. What is Amazon S3?

Answer:

Amazon S3 is an object storage service used for storing files, backups, logs, and static website content. It offers high durability, scalability, and security. In my project, S3 stores Terraform remote state files and application backups.

Q14. Difference between S3, EBS, and EFS?

Answer:

S3 is object storage for files and backups. EBS is block storage attached to EC2 instances for operating systems and applications. EFS is a shared file storage service that multiple EC2 instances can access simultaneously. Each service is designed for different storage requirements.

Q15. What is Amazon EBS?

Answer:

Amazon EBS (Elastic Block Store) provides persistent block storage for EC2 instances. It is commonly used to store operating systems, applications, and databases. EBS volumes remain available even after an EC2 instance is stopped, making them suitable for production environments.

Q16. What is Route 53?

Answer:

Amazon Route 53 is AWS's managed DNS service. It routes user requests to applications using domain names and supports health checks, failover routing, and weighted routing. In my project, Route 53 directs internet traffic to the Application Load Balancer.

Q17. What is CloudWatch?

Answer:

Amazon CloudWatch is a monitoring service that collects metrics, logs, and events from AWS resources. It helps monitor system performance, create dashboards, and configure alarms. In my project, CloudWatch is used to monitor EC2, EKS, and application logs.

Q18. What is CloudTrail?

Answer:

AWS CloudTrail records all AWS API calls and account activities for auditing and security. It helps identify who performed an action, when it occurred, and from where. We use CloudTrail for compliance, troubleshooting, and security investigations.

Q19. What is the Principle of Least Privilege?

Answer:

The Principle of Least Privilege means granting only the minimum permissions required to perform a task. This reduces security risks and prevents unauthorized access. In my project, IAM Roles are assigned only the permissions required by EC2, EKS, and Jenkins.

Q20. Difference between IAM User and IAM Role?

Answer:

An IAM User represents a person or application with long-term credentials. An IAM Role provides temporary credentials that users or AWS services can assume when needed. In my

project, we use IAM Roles for EC2, EKS, and Jenkins instead of long-term access keys to improve security.

Chapter 2: Terraform Interview Questions (Q21–Q40)

Q21. What is Terraform?

Answer:

Terraform is an Infrastructure as Code (IaC) tool used to provision and manage cloud infrastructure through code. It automates resource creation, ensures consistency, and supports version control. In my project, I use Terraform to create VPCs, EKS clusters, IAM roles, security groups, and other AWS resources.

Q22. What are the advantages of Terraform?

Answer:

Terraform provides Infrastructure as Code, automation, version control, consistency, and multi-cloud support. It reduces manual errors and allows infrastructure to be recreated easily. In my project, Terraform helps us provision AWS infrastructure quickly and maintain the same configuration across development, UAT, and production environments.

Q23. What is a Terraform State File?

Answer:

The Terraform state file stores information about the infrastructure managed by Terraform. It maps Terraform configurations to actual cloud resources and helps determine what changes are required during the next deployment. In my project, the state file is stored remotely in Amazon S3.

Q24. Why do you use a Remote Backend?

Answer:

A remote backend stores the Terraform state file centrally, enabling team collaboration and preventing state conflicts. In my project, we use Amazon S3 for remote state storage and DynamoDB for state locking to ensure only one person can modify the infrastructure at a time.

Q25. Why is DynamoDB used with Terraform?

Answer:

DynamoDB is used for state locking when Terraform stores its state in Amazon S3. It prevents multiple users from modifying the infrastructure simultaneously, avoiding state corruption. This ensures safe collaboration in team environments.

Q26. What is a Terraform Module?

Answer:

A Terraform module is a reusable collection of Terraform resources. Modules help standardize infrastructure, reduce code duplication, and simplify maintenance. In my project, separate modules are created for VPC, EKS, IAM, ALB, and security groups.

Q27. What are Input Variables?

Answer:

Input variables make Terraform configurations reusable by allowing values to be passed during execution. Instead of hardcoding values, variables are defined once and reused across environments. In my project, variables are used for AWS region, VPC CIDR, instance types, and cluster names.

Q28. What are Output Variables?

Answer:

Output variables display important information after Terraform creates infrastructure. They allow values such as VPC IDs, subnet IDs, ALB DNS names, or EKS cluster endpoints to be shared with other modules or users.

Q29. What is the Terraform Lifecycle?

Answer:

The Terraform lifecycle includes `terraform init`, `terraform validate`, `terraform plan`, `terraform apply`, and `terraform destroy`. This workflow initializes the project, validates the configuration, previews changes, provisions infrastructure, and removes resources when no longer needed.

Q30. What does `terraform init` do?

Answer:

`terraform init` initializes the Terraform working directory. It downloads the required providers, configures the backend, and installs modules. It is the first command executed before planning or applying infrastructure changes.

Q31. What does `terraform plan` do?

Answer:

`terraform plan` compares the current infrastructure with the Terraform configuration and displays the changes that will be made. It helps review additions, updates, or deletions before applying them, reducing the risk of unintended changes.

Q32. What does `terraform apply` do?

Answer:

`terraform apply` executes the changes shown in the Terraform plan and provisions or updates the infrastructure. After user confirmation, Terraform creates, modifies, or deletes AWS resources to match the desired configuration.

Q33. What does `terraform destroy` do?

Answer:

`terraform destroy` removes all infrastructure managed by the Terraform configuration. It is commonly used to clean up development or testing environments and avoid unnecessary cloud costs.

Q34. What is Terraform Drift?

Answer:

Terraform drift occurs when cloud resources are modified manually outside Terraform. During `terraform plan`, Terraform detects differences between the actual infrastructure and the state file, allowing administrators to restore the desired configuration.

Q35. What are Terraform Workspaces?

Answer:

Terraform Workspaces allow the same configuration to manage multiple environments such as development, UAT, and production while maintaining separate state files. This reduces code duplication and simplifies environment management.

Q36. What is the difference between `count` and `for_each`?

Answer:

`count` creates multiple identical resources using numeric indexes, while `for_each` creates resources using unique keys from a map or set. I prefer `for_each` when resources require unique names or configurations.

Q37. What is the `depends_on` argument?

Answer:

`depends_on` explicitly defines resource dependencies when Terraform cannot determine them automatically. It ensures resources are created in the correct order, preventing deployment failures caused by missing dependencies.

Q38. What are Terraform Provisioners?

Answer:

Provisioners execute scripts or commands on local or remote machines after resource creation. Common provisioners include `local-exec` and `remote-exec`. They should be used sparingly because Terraform recommends using configuration management tools like Ansible whenever possible.

Q39. What is the difference between `terraform refresh` and `terraform plan`?

Answer:

`terraform refresh` updates the state file with the current infrastructure status. `terraform plan` compares the updated state with the Terraform configuration and shows what changes are required to achieve the desired state.

Q40. How do you organize Terraform code in your project?

Answer:

I follow a modular structure with separate modules for VPC, EKS, IAM, ALB, and security groups. Environment-specific configurations are stored in separate folders for development, UAT, and production. The state file is stored in Amazon S3 with DynamoDB state locking for secure team collaboration.

Chapter 3: Git & GitHub Interview Questions (Q41–Q60)

Q41. What is Git?

Answer:

Git is a distributed version control system used to track source code changes and enable team collaboration. It maintains the complete history of code changes, making it easy to manage branches, merge code, and roll back changes. In my project, we use Git with GitHub for source code management.

Q42. What is GitHub?

Answer:

GitHub is a cloud-based platform that hosts Git repositories. It provides features such as pull requests, branch protection, code reviews, issue tracking, and GitHub Actions for CI/CD. In my project, developers push code to GitHub, which triggers our deployment pipeline.

Q43. Explain the Git workflow.

Answer:

The Git workflow starts with creating a feature branch, making code changes, committing locally, and pushing to the remote repository. A Pull Request is created for code review. After approval, the changes are merged into the main branch, triggering the CI/CD pipeline.

Q44. What is GitFlow?

Answer:

GitFlow is a branching strategy using **main**, **develop**, **feature**, **release**, and **hotfix** branches. It helps organize development and releases. In my project, developers create feature branches, merge them into develop, and release code after successful testing.

Q45. What is Trunk-Based Development?

Answer:

Trunk-Based Development is a branching strategy where developers frequently commit small changes to the main branch or short-lived feature branches. It reduces merge conflicts and supports continuous integration with faster delivery.

Q46. Difference between Git Merge and Git Rebase?

Answer:

Git Merge combines two branches and preserves the complete commit history by creating a merge commit. Git Rebase moves commits onto another branch, creating a cleaner, linear history. We generally use merge for shared branches and rebase before creating pull requests.

Q47. What is a Pull Request (PR)?

Answer:

A Pull Request is a request to merge code from one branch into another. It enables code review, discussion, automated testing, and approval before merging. In my project, every code change goes through a Pull Request to maintain code quality.

Q48. What is Git Stash?

Answer:

Git Stash temporarily saves uncommitted changes without creating a commit. It allows developers to switch branches, perform urgent work, and later restore the saved changes using `git stash pop`.

Q49. What is Git Cherry-pick?

Answer:

Git Cherry-pick copies a specific commit from one branch to another without merging the entire branch. It is useful when only one bug fix or feature needs to be moved to another branch.

Q50. What is a Merge Conflict?

Answer:

A merge conflict occurs when Git cannot automatically merge changes because the same file or lines were modified in different branches. The conflict must be resolved manually, tested, and then committed before completing the merge.

Q51. Difference between Git Fetch and Git Pull?

Answer:

`git fetch` downloads the latest changes from the remote repository without merging them. `git pull` downloads and automatically merges the changes into the current branch. I usually run `git fetch` first to review incoming changes before merging.

Q52. What is Git Clone?

Answer:

`git clone` creates a complete local copy of a remote repository, including all branches and commit history. It is the first command used when working on a new project.

Q53. What is Git Commit?

Answer:

A Git commit records a snapshot of code changes in the local repository. Each commit has a unique ID and message, making it easy to track changes and revert if needed.

Q54. What is Branch Protection?

Answer:

Branch protection prevents direct changes to important branches like `main` or `master`. It enforces pull requests, code reviews, status checks, and approval policies before allowing code to be merged.

Q55. How do you resolve merge conflicts?

Answer:

I identify the conflicting files, review both changes, manually resolve the conflicts, test the application, and commit the resolved code. Finally, I push the updated branch and complete the merge or pull request.

Q56. What is `.gitignore`?

Answer:

The `.gitignore` file specifies files and directories that Git should not track, such as log files, temporary files, build artifacts, and sensitive configuration files. This keeps the repository clean and secure.

Q57. What is a Tag in Git?

Answer:

A Git tag marks a specific commit, usually for application releases like `v1.0` or `v2.1`. Tags make it easy to identify and deploy stable versions of the application.

Q58. How do you revert a commit?

Answer:

`git revert` creates a new commit that reverses the changes made by a previous commit without changing the project's history. It is commonly used in shared repositories because it is safe and preserves the commit history.

Q59. What is GitHub Actions?

Answer:

GitHub Actions is GitHub's CI/CD service used to automate build, test, and deployment workflows. In my project, it builds Docker images, scans them with Trivy, pushes them to Amazon ECR, and triggers deployments to Amazon EKS through ArgoCD.

Q60. Explain your GitHub Actions workflow.

Answer:

When developers push code to GitHub, GitHub Actions automatically starts the pipeline. It checks out the code, runs tests, builds the Docker image, scans it with Trivy, pushes the image to Amazon ECR, updates the Helm chart, and ArgoCD deploys the application to Amazon EKS.

Chapter 4: Docker Interview Questions (Q61–Q80)

Q61. What is Docker?

Answer:

Docker is a containerization platform that packages an application and its dependencies into a container. Containers run consistently across different environments, making deployment faster and more reliable. In my project, we use Docker to containerize applications before deploying them to Amazon EKS.

Q62. What is the difference between a Virtual Machine and a Docker Container?

Answer:

A Virtual Machine includes a complete operating system and uses more resources. A Docker container shares the host operating system kernel, making it lightweight, faster, and more efficient. Containers start quickly and are ideal for microservices.

Q63. What is a Docker Image?

Answer:

A Docker image is a read-only template containing the application, libraries, dependencies, and runtime required to run a container. Images are created using a Dockerfile and stored in registries such as Amazon ECR or Docker Hub.

Q64. What is a Docker Container?

Answer:

A Docker container is a running instance of a Docker image. It provides an isolated environment where the application executes. Multiple containers can run from the same image while remaining independent of each other.

Q65. What is a Dockerfile?

Answer:

A Dockerfile is a text file containing instructions to build a Docker image. It defines the base image, copies application files, installs dependencies, exposes ports, and specifies the startup command. In my project, every application has its own Dockerfile.

Q66. Explain the Docker lifecycle.

Answer:

The Docker lifecycle starts with writing a Dockerfile, building an image using `docker build`, storing it in a registry like Amazon ECR, pulling the image to the target server, and running it as a container using `docker run`.

Q67. What is the difference between COPY and ADD?

Answer:

`COPY` copies files from the local system into the Docker image. `ADD` provides additional features such as extracting compressed files and downloading content from URLs. In most cases, `COPY` is recommended because it is simpler and more predictable.

Q68. What is the difference between CMD and ENTRYPOINT?

Answer:

`CMD` provides the default command executed when a container starts and can be overridden.

`ENTRYPOINT` defines the main executable and always runs unless explicitly replaced. They are often used together to provide default arguments.

Q69. What is a Docker Registry?

Answer:

A Docker Registry stores and manages Docker images. Common registries include Docker Hub, Amazon ECR, and JFrog Artifactory. In my project, Docker images are stored in Amazon ECR after being built through the CI/CD pipeline.

Q70. What is Amazon ECR?

Answer:

Amazon Elastic Container Registry (ECR) is a managed Docker image repository provided by AWS. It securely stores container images and integrates with Amazon EKS, ECS, and IAM for authentication and access control.

Q71. What is a Docker Volume?

Answer:

A Docker Volume provides persistent storage for containers. Data stored in a volume remains available even if the container is deleted. Volumes are commonly used for databases, logs, and application data that must survive container restarts.

Q72. What is Bind Mount?

Answer:

A bind mount maps a directory from the host machine directly into a container. It is commonly used during development so code changes on the host are immediately reflected inside the container without rebuilding the image.

Q73. What is Docker Networking?

Answer:

Docker networking enables communication between containers and external systems. Docker supports Bridge, Host, Overlay, Macvlan, and None network drivers. The Bridge network is the default and is commonly used for standalone containers.

Q74. What is Docker Compose?

Answer:

Docker Compose is a tool used to define and manage multi-container applications using a `docker-compose.yml` file. It allows multiple services, networks, and volumes to be started together with a single command.

Q75. What is a Multi-stage Docker Build?

Answer:

A multi-stage build uses multiple `FROM` statements in a Dockerfile to separate the build environment from the runtime environment. It reduces image size, removes unnecessary build tools, and improves security.

Q76. How do you reduce Docker image size?

Answer:

I use lightweight base images like Alpine, multi-stage builds, combine RUN commands, remove unnecessary packages, clean temporary files, and use `.dockerignore` to exclude unwanted files. This results in smaller, faster, and more secure images.

Q77. How do you troubleshoot a container that is not starting?

Answer:

I check the container status using `docker ps -a`, review logs with `docker logs`, inspect the container configuration, verify environment variables, check port conflicts, and ensure the application starts correctly inside the container.

Q78. What is the difference between `docker stop` and `docker kill`?

Answer:

`docker stop` gracefully terminates a container by sending a SIGTERM signal before forcing it to stop. `docker kill` immediately terminates the container using SIGKILL without allowing the application to shut down gracefully.

Q79. What is `.dockerignore`?

Answer:

The `.dockerignore` file specifies files and directories that should not be included in the Docker build context. It reduces image size, improves build performance, and prevents sensitive or unnecessary files from being copied into the image.

Q80. Explain your Docker workflow in the project.

Answer:

Developers commit code to GitHub, triggering GitHub Actions. The pipeline builds a Docker image, scans it with Trivy, pushes it to Amazon ECR, updates the Helm chart, and ArgoCD deploys the image to Amazon EKS. This provides automated and consistent deployments.

Chapter 5: Kubernetes Interview Questions (Q81–Q100)

Q81. What is Kubernetes?

Answer:

Kubernetes is an open-source container orchestration platform used to deploy, manage, scale, and monitor containerized applications. It provides features such as self-healing, load balancing, rolling updates, and auto-scaling. In my project, we use Amazon EKS to manage Kubernetes clusters.

Q82. What is Amazon EKS?

Answer:

Amazon EKS (Elastic Kubernetes Service) is AWS's managed Kubernetes service. AWS manages the control plane, while we manage worker nodes and applications. In my project, we use EKS to deploy containerized applications with high availability and integrate it with ALB, IAM, and ECR.

Q83. What is a Pod?

Answer:

A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers that share the same network and storage. In most cases, a Pod runs a single application container and is managed by a Deployment.

Q84. What is a Deployment?

Answer:

A Deployment manages Pods and ReplicaSets. It ensures the desired number of Pods are running and supports rolling updates, rollbacks, and scaling. In my project, Deployments are used to manage frontend and backend applications.

Q85. What is a ReplicaSet?

Answer:

A ReplicaSet ensures that the desired number of identical Pods are always running. If a Pod fails, Kubernetes automatically creates a new one. ReplicaSets are typically managed by Deployments rather than created directly.

Q86. Difference between Pod and Deployment?

Answer:

A Pod is the smallest unit that runs one or more containers. A Deployment manages multiple Pods, handles rolling updates, scaling, and self-healing. In production, applications are deployed using Deployments rather than standalone Pods.

Q87. What is a Namespace?

Answer:

A Namespace is used to logically separate resources within a Kubernetes cluster. It helps organize applications, manage access, and apply resource quotas. In my project, we use separate namespaces for development, UAT, and production environments.

Q88. What is a Service in Kubernetes?

Answer:

A Service provides a stable network endpoint to access Pods. Since Pod IP addresses change, Services ensure reliable communication. Kubernetes supports ClusterIP, NodePort, LoadBalancer, and ExternalName service types.

Q89. What is ClusterIP?

Answer:

ClusterIP is the default Kubernetes Service type. It exposes an application only within the cluster and is used for communication between internal services such as frontend, backend, and databases.

Q90. What is NodePort?

Answer:

NodePort exposes an application on a static port of every Kubernetes node. Users can access the application using the node's IP address and the assigned port. It is mainly used for testing or non-production environments.

Q91. What is a LoadBalancer Service?

Answer:

A LoadBalancer Service creates an external cloud load balancer, such as an AWS Application Load Balancer or Network Load Balancer, to expose applications to the internet. It is commonly used for production workloads.

Q92. What is Ingress?

Answer:

Ingress manages external HTTP and HTTPS access to applications. It supports host-based routing, path-based routing, SSL termination, and load balancing. In my project, we use AWS ALB Ingress Controller to expose applications running on EKS.

Q93. What is a ConfigMap?

Answer:

A ConfigMap stores non-sensitive configuration data such as URLs, environment variables, and application settings. It separates configuration from application code, making deployments easier to manage across environments.

Q94. What is a Secret?

Answer:

A Secret stores sensitive information such as database passwords, API keys, and TLS certificates. Secrets are mounted as environment variables or volumes and help avoid hardcoding sensitive values in application code.

Q95. Difference between ConfigMap and Secret?

Answer:

A ConfigMap stores non-sensitive configuration data, while a Secret stores sensitive information such as passwords and API keys. In my project, ConfigMaps manage application settings, and Secrets securely store database credentials.

Q96. What is a Persistent Volume (PV)?

Answer:

A Persistent Volume is a storage resource in Kubernetes that provides persistent data independent of a Pod's lifecycle. It is commonly backed by AWS EBS or EFS in cloud environments.

Q97. What is a Persistent Volume Claim (PVC)?

Answer:

A Persistent Volume Claim is a request for storage made by a Pod. Kubernetes automatically binds the claim to a suitable Persistent Volume, allowing applications to use persistent storage without knowing the underlying storage details.

Q98. Difference between PV and PVC?

Answer:

A Persistent Volume represents the actual storage resource, while a Persistent Volume Claim is the request made by an application for storage. Kubernetes binds the PVC to an available PV based on the requested capacity and access mode.

Q99. What is RBAC?

Answer:

RBAC (Role-Based Access Control) controls user and service account permissions in Kubernetes. It uses Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings to grant access. In my project, RBAC ensures developers have access only to their assigned namespaces.

Q100. What is a Service Account?

Answer:

A Service Account provides an identity for Pods to communicate securely with the Kubernetes API or AWS services. In Amazon EKS, we use IAM Roles for Service Accounts (IRSA) so Pods can access AWS resources without storing credentials.

Chapter 5: Kubernetes Interview Questions (Q101–Q120)

Q101. What is a StatefulSet?

Answer:

A StatefulSet manages stateful applications like MySQL, PostgreSQL, and MongoDB. It provides stable Pod names, persistent storage, and ordered deployment or termination. Unlike Deployments, each Pod has its own identity and storage.

Q102. What is a DaemonSet?

Answer:

A DaemonSet ensures one Pod runs on every worker node. It is commonly used for monitoring, logging, and security agents. In my project, Node Exporter and Fluent Bit run as DaemonSets to collect metrics and logs from all nodes.

Q103. What is a Job?

Answer:

A Job runs a task until it completes successfully. It is mainly used for one-time tasks such as database migration, backups, or batch processing. Once the task finishes successfully, the Job exits.

Q104. What is a CronJob?

Answer:

A CronJob schedules Jobs at specific times using cron expressions. It is used for recurring tasks like backups, report generation, log cleanup, or database maintenance.

Q105. What is Horizontal Pod Autoscaler (HPA)?

Answer:

HPA automatically increases or decreases the number of Pods based on CPU, memory, or custom metrics. It helps applications handle varying traffic loads while optimizing resource usage. In my project, HPA scales application Pods based on CPU utilization.

Q106. What is Vertical Pod Autoscaler (VPA)?

Answer:

VPA automatically adjusts CPU and memory requests and limits for Pods based on resource usage. It improves resource utilization but is generally not used together with HPA on the same workload.

Q107. What are Liveness and Readiness Probes?

Answer:

A Liveness Probe checks whether an application is healthy. If it fails, Kubernetes restarts the container. A Readiness Probe checks whether the application is ready to receive traffic. If it fails, traffic is temporarily stopped until the Pod becomes ready.

Q108. What is a Rolling Update?

Answer:

A Rolling Update gradually replaces old Pods with new ones without downtime. Kubernetes ensures enough healthy Pods remain available during deployment. This is the default deployment strategy used in most production environments.

Q109. What is a Rollback?

Answer:

A Rollback restores the previous stable version if a deployment fails. Kubernetes supports rollback using Deployment history. In my project, we also use ArgoCD to revert to the previous Git commit when necessary.

Q110. What are Taints and Tolerations?

Answer:

Taints prevent Pods from being scheduled on specific nodes, while Tolerations allow eligible Pods to run on those nodes. They are commonly used to dedicate nodes for system workloads or high-priority applications.

Q111. What is Node Affinity?

Answer:

Node Affinity controls where Pods are scheduled based on node labels. It ensures workloads run only on suitable nodes, such as nodes with SSD storage or GPU support.

Q112. What is Pod Affinity and Anti-Affinity?

Answer:

Pod Affinity schedules Pods close to each other for better communication, while Pod Anti-Affinity spreads Pods across different nodes to improve high availability and fault tolerance.

Q113. What is CrashLoopBackOff?

Answer:

CrashLoopBackOff occurs when a container repeatedly starts and crashes. Common causes include application errors, incorrect configuration, missing Secrets, or database connectivity issues. I troubleshoot using `kubectl logs` and `kubectl describe pod`.

Q114. What is ImagePullBackOff?

Answer:

ImagePullBackOff occurs when Kubernetes cannot download a container image. Common reasons include incorrect image names, invalid tags, authentication failures, or registry connectivity issues. I verify the image name, registry access, and imagePullSecrets.

Q115. How do you troubleshoot a Pending Pod?

Answer:

I check `kubectl describe pod` for scheduling events, verify CPU and memory availability, review node status, check PVC binding, node taints, and affinity rules. These are the most common reasons why a Pod remains in the Pending state.

Q116. How do you troubleshoot a NotReady node?

Answer:

I check the node status, kubelet service, CPU and memory usage, disk space, network connectivity, and system logs. I also verify whether the node can communicate with the Kubernetes control plane.

Q117. How do you troubleshoot an application that is not accessible?

Answer:

I verify the Pod status, Deployment, Service, Ingress, ALB health checks, DNS configuration, Security Groups, application logs, and monitoring dashboards. This systematic approach helps identify whether the issue is with networking, Kubernetes, or the application itself.

Q118. How do you scale an application in Kubernetes?

Answer:

Applications can be scaled manually using `kubectl scale` or automatically using HPA. In production, I prefer HPA because it adjusts the number of Pods automatically based on CPU or memory utilization.

Q119. Explain your Kubernetes deployment process.

Answer:

Developers push code to GitHub, triggering GitHub Actions. The pipeline builds the Docker image, scans it with Trivy, pushes it to Amazon ECR, updates the Helm chart, and ArgoCD deploys the application to Amazon EKS using a rolling update strategy.

Q120. Explain your EKS architecture.

Answer:

Our EKS cluster runs across multiple Availability Zones within a VPC. Worker nodes are deployed in private subnets, while the ALB is placed in public subnets. Terraform provisions the infrastructure, GitHub Actions manages CI, ArgoCD handles CD, and Prometheus with Grafana monitors the applications.

Chapter 6: CI/CD, Jenkins, GitHub Actions, Helm, ArgoCD & Ansible (Q121–Q150)

Q121. What is CI/CD?

Answer:

CI/CD stands for Continuous Integration and Continuous Deployment/Delivery. CI automatically builds and tests code after every commit, while CD automates application deployment. In my project, GitHub Actions handles CI, and ArgoCD deploys applications to Amazon EKS using GitOps.

Q122. What is Jenkins?

Answer:

Jenkins is an open-source automation server used to build, test, and deploy applications. It supports pipelines, plugins, and integrations with Git, Docker, Kubernetes, and AWS. I have used Jenkins to automate Docker image builds and deployments.

Q123. What is GitHub Actions?

Answer:

GitHub Actions is GitHub's built-in CI/CD service that automates software workflows. In my project, it checks out code, builds Docker images, scans them with Trivy, pushes them to Amazon ECR, and updates the GitOps repository.

Q124. Explain your CI/CD pipeline.

Answer:

Developers push code to GitHub, triggering GitHub Actions. The pipeline builds the application, creates a Docker image, scans it with Trivy, pushes it to Amazon ECR, updates the Helm chart, and ArgoCD automatically deploys the latest version to Amazon EKS.

Q125. What is a Jenkins Pipeline?

Answer:

A Jenkins Pipeline is a series of automated stages that define the build, test, and deployment process. Pipelines are written in Groovy using a Jenkinsfile and support version control, automation, and repeatable deployments.

Q126. Difference between Declarative and Scripted Pipeline?

Answer:

Declarative Pipelines use a simple predefined syntax and are easier to maintain. Scripted Pipelines provide greater flexibility using Groovy code but are more complex. Declarative Pipelines are commonly preferred for standard CI/CD workflows.

Q127. What is a Jenkins Agent?

Answer:

A Jenkins Agent is a machine that executes build jobs assigned by the Jenkins Controller. Agents improve scalability by allowing multiple pipelines to run in parallel on different systems.

Q128. What is Helm?

Answer:

Helm is a package manager for Kubernetes that simplifies application deployment using reusable Helm Charts. It helps manage application versions, templates, and configuration across multiple environments.

Q129. What is a Helm Chart?

Answer:

A Helm Chart is a package containing Kubernetes manifests, templates, and configuration files required to deploy an application. It allows reusable and parameterized deployments across development, testing, and production environments.

Q130. What is `values.yaml`?

Answer:

`values.yaml` stores configurable values used by Helm templates, such as image names, replica counts, resource limits, and environment variables. It allows the same Helm Chart to be reused across multiple environments.

Q131. What is ArgoCD?

Answer:

ArgoCD is a GitOps continuous delivery tool for Kubernetes. It continuously monitors a Git repository and automatically synchronizes Kubernetes clusters with the desired state stored in Git.

Q132. What is GitOps?

Answer:

GitOps is a deployment approach where Git serves as the single source of truth for infrastructure and application configurations. Tools like ArgoCD continuously synchronize Kubernetes clusters with changes committed to Git.

Q133. What are the advantages of ArgoCD?

Answer:

ArgoCD provides automated deployments, easy rollbacks, continuous synchronization, version-controlled deployments, improved visibility, and drift detection. It simplifies Kubernetes deployments while ensuring the cluster always matches the Git repository.

Q134. Explain your GitOps workflow.

Answer:

GitHub Actions builds and pushes the Docker image to Amazon ECR, then updates the image tag in the GitOps repository. ArgoCD detects the Git change and automatically synchronizes the updated application to the Amazon EKS cluster.

Q135. What is JFrog Artifactory?

Answer:

JFrog Artifactory is an artifact repository used to store binaries, libraries, Docker images, and build artifacts. It integrates with CI/CD pipelines and improves dependency management, security, and artifact versioning.

Q136. What is Ansible?

Answer:

Ansible is an open-source configuration management and automation tool. It automates software installation, server configuration, and application deployment using YAML-based playbooks without requiring agents on target systems.

Q137. What is an Ansible Playbook?

Answer:

An Ansible Playbook is a YAML file containing automation tasks executed on managed hosts. It defines the desired system configuration and helps automate repetitive administrative tasks consistently.

Q138. What is Inventory in Ansible?

Answer:

An Inventory is a list of managed hosts grouped by environment or function. It can be static or dynamic and allows playbooks to target specific servers efficiently.

Q139. What is Idempotency in Ansible?

Answer:

Idempotency means running the same playbook multiple times produces the same result without making unnecessary changes. This ensures safe, predictable, and repeatable infrastructure automation.

Q140. Explain Blue-Green Deployment.

Answer:

Blue-Green Deployment maintains two identical environments. The new version is deployed to the inactive environment, tested, and traffic is switched after validation. This minimizes downtime and provides quick rollback if needed.

Q141. What is a Rolling Deployment?

Answer:

A Rolling Deployment gradually replaces old application instances with new ones while keeping the application available. Kubernetes performs rolling updates automatically to achieve zero or minimal downtime.

Q142. How do you perform a rollback?

Answer:

If a deployment fails, I use Kubernetes rollout undo, Helm rollback, or revert the Git commit in ArgoCD. Before rollback, I verify logs, identify the root cause, and confirm application health after restoration.

Q143. How do you secure a CI/CD pipeline?

Answer:

I store credentials in GitHub Secrets or Jenkins Credentials, enforce RBAC, scan Docker images with Trivy, use IAM Roles instead of access keys, enable branch protection, and restrict production deployments through approval workflows.

Q144. What is Trivy?

Answer:

Trivy is an open-source security scanner for Docker images, Kubernetes configurations, and source code. In my project, every Docker image is scanned before being pushed to Amazon ECR to detect vulnerabilities.

Q145. What happens if a pipeline fails?

Answer:

I identify the failed stage, review logs, verify recent code changes, fix the issue, rerun the pipeline, and validate the deployment. If production is affected, I perform a rollback and conduct a root cause analysis.

Q146. How do you manage different environments?

Answer:

I maintain separate configurations for development, UAT, and production using Terraform variables, Helm values files, Kubernetes namespaces, and Git branches. This ensures consistent deployments while keeping environment-specific settings separate.

Q147. What is Continuous Delivery vs Continuous Deployment?

Answer:

Continuous Delivery automates build, test, and deployment but requires manual approval before production. Continuous Deployment automatically releases every successful change to production without manual intervention.

Q148. What is Infrastructure as Code (IaC)?

Answer:

Infrastructure as Code manages infrastructure using configuration files instead of manual processes. It improves consistency, automation, version control, and repeatability. In my project, Terraform is used to provision AWS infrastructure.

Q149. Explain your deployment strategy.

Answer:

We use GitHub Actions for CI, Amazon ECR for image storage, Helm for packaging, ArgoCD for GitOps deployment, and Amazon EKS for hosting applications. Deployments use rolling updates with monitoring through Prometheus and Grafana.

Q150. Explain your end-to-end DevOps workflow.

Answer:

Developers push code to GitHub, triggering GitHub Actions. The pipeline builds the application, creates a Docker image, scans it with Trivy, pushes it to Amazon ECR, updates the GitOps repository, and ArgoCD deploys it to Amazon EKS. Terraform manages infrastructure, while Prometheus, Grafana, and CloudWatch provide monitoring and alerting.

Chapter 7: Linux, Monitoring, Production Support & HR (Q151–Q180)

Q151. What is Linux?

Answer:

Linux is an open-source operating system widely used for servers and cloud environments. It provides stability, security, and excellent performance. In my project, I use Linux to manage EC2 instances, troubleshoot applications, and automate tasks using shell scripting.

Q152. What are the Linux commands you use daily?

Answer:

I frequently use `ls`, `cd`, `pwd`, `cp`, `mv`, `rm`, `cat`, `grep`, `find`, `chmod`, `chown`, `df -h`, `free -m`, `top`, `ps -ef`, `netstat`, `curl`, `systemctl`, `journalctl`, and `tail -f` for administration and troubleshooting.

Q153. Difference between `chmod 777` and `chmod 755`?

Answer:

`chmod 777` gives read, write, and execute permissions to everyone, making it insecure. `chmod 755` gives full permissions to the owner and read-execute permissions to others. In production, I prefer `755` or more restrictive permissions for security.

Q154. What is Prometheus?

Answer:

Prometheus is an open-source monitoring tool that collects time-series metrics from applications and infrastructure. It stores metrics and allows querying using PromQL. In my project, Prometheus monitors Kubernetes clusters and application performance.

Q155. What is Grafana?

Answer:

Grafana is a visualization platform used to create dashboards for monitoring metrics collected from Prometheus, CloudWatch, and other sources. We use Grafana to monitor CPU, memory, pod health, and application performance.

Q156. What is PromQL?

Answer:

PromQL is the query language used by Prometheus to retrieve and analyze metrics. It helps create dashboards, generate alerts, and monitor application performance through Grafana.

Q157. What is CloudWatch?

Answer:

Amazon CloudWatch is AWS's monitoring service for collecting metrics, logs, and events. It helps create dashboards, configure alarms, and monitor AWS resources. We use CloudWatch for EC2, EKS, and application log monitoring.

Q158. How do you monitor production applications?

Answer:

We use Prometheus to collect metrics, Grafana for dashboards, CloudWatch for AWS monitoring, and application logs for troubleshooting. Alerts are configured for high CPU, memory usage, pod failures, and application downtime.

Q159. What is an Alert?

Answer:

An alert is a notification generated when a predefined condition is met, such as high CPU usage or application failure. Alerts help teams respond quickly before issues impact users.

Q160. Explain your monitoring strategy.

Answer:

We monitor infrastructure using CloudWatch, Kubernetes using Prometheus, and visualize metrics in Grafana. Alerts are configured for CPU, memory, disk usage, pod health, node availability, and application response time to ensure quick issue detection.

Q161. How do you troubleshoot a production issue?

Answer:

I first understand the impact, check monitoring dashboards, review logs, identify the root cause, implement a fix or rollback if needed, verify application health, and document the incident with preventive actions.

Q162. A Pod is in CrashLoopBackOff. What will you do?

Answer:

I check `kubectl describe pod` and `kubectl logs` to identify the cause. Then I verify application configuration, Secrets, ConfigMaps, database connectivity, resource limits, and startup commands before redeploying the application.

Q163. An application is not accessible. What will you check?

Answer:

I verify the Pod status, Deployment, Service, Ingress, ALB health checks, DNS records, Security Groups, and application logs. This helps determine whether the issue is related to networking, Kubernetes, or the application.

Q164. EC2 CPU utilization is 100%. What will you do?

Answer:

I identify the process consuming CPU using Linux tools, review application logs, analyze monitoring dashboards, and determine whether the issue is due to traffic, inefficient code, or resource shortages. If needed, I scale the application using Auto Scaling or HPA.

Q165. How do you perform Root Cause Analysis (RCA)?

Answer:

I collect logs, metrics, and events, identify the root cause, document the timeline, implement corrective actions, and define preventive measures to avoid recurrence. The RCA is shared with stakeholders after service restoration.

Q166. What is a Sev-1 Incident?

Answer:

A Sev-1 incident is a critical production issue causing major business impact or complete service outage. It requires immediate response, stakeholder communication, service restoration, and a detailed Root Cause Analysis after resolution.

Q167. How do you handle production deployments?

Answer:

I verify the deployment package, ensure approvals are complete, take backups if required, deploy using the CI/CD pipeline, monitor application health, validate functionality, and keep a rollback plan ready in case of issues.

Q168. What is a Change Request (CR)?

Answer:

A Change Request is a formal approval process for production changes. It includes implementation steps, risk assessment, rollback plans, testing evidence, and approval from stakeholders before deployment.

Q169. Explain your production support responsibilities.

Answer:

My responsibilities include monitoring production systems, troubleshooting issues, supporting deployments, resolving incidents, performing RCA, collaborating with development teams, maintaining CI/CD pipelines, and ensuring high application availability.

Q170. Explain your current project.

Answer:

My current project is a three-tier application deployed on Amazon EKS. Terraform provisions AWS infrastructure, GitHub Actions manages CI, ArgoCD performs GitOps deployments, Amazon ECR stores Docker images, and Prometheus with Grafana monitors the application.

Q171. Tell me about a challenging issue you resolved.

Answer:

We experienced application downtime after a deployment due to an incorrect database configuration. I identified the issue through Kubernetes logs, corrected the configuration, redeployed the application, restored service, and documented the RCA to prevent recurrence.

Q172. Why do you want to change your job?

Answer:

I am looking for opportunities to work on larger cloud environments, learn modern DevOps technologies, contribute to challenging projects, and continue growing professionally while adding value to the organization.

Q173. Why should we hire you?

Answer:

I have 12 years of IT experience, including 5+ years in Cloud and DevOps. I have hands-on expertise in AWS, Terraform, Docker, Kubernetes, CI/CD, GitHub Actions, and production support. I enjoy automation, problem-solving, and delivering reliable cloud solutions.

Q174. What are your strengths?

Answer:

My strengths include troubleshooting production issues, infrastructure automation, continuous learning, teamwork, and delivering reliable cloud solutions. I remain calm during incidents and focus on resolving issues efficiently.

Q175. What is your weakness?

Answer:

Earlier, I focused mainly on technical execution. I now actively improve my documentation, knowledge sharing, and communication to ensure better collaboration across teams.

Q176. Where do you see yourself in five years?

Answer:

I see myself as a Senior Cloud or Platform Engineer leading cloud automation initiatives, mentoring team members, designing scalable architectures, and contributing to modern platform engineering practices.

Q177. What are your daily responsibilities?

Answer:

I provision infrastructure using Terraform, manage Kubernetes deployments, maintain CI/CD pipelines, monitor applications, troubleshoot production issues, support developers, review infrastructure changes, and participate in production releases.

Q178. Explain your end-to-end DevOps workflow.

Answer:

Developers push code to GitHub, triggering GitHub Actions. The pipeline builds Docker images, scans them with Trivy, pushes them to Amazon ECR, updates Helm charts, and ArgoCD deploys the application to Amazon EKS. Terraform manages infrastructure, and Prometheus, Grafana, and CloudWatch provide monitoring.

Q179. What is your biggest achievement?

Answer:

One of my key achievements was automating AWS infrastructure using Terraform and implementing a GitHub Actions and ArgoCD-based CI/CD pipeline. This reduced manual deployment effort, improved consistency, and accelerated application releases.

Q180. Do you have any questions for us?

Answer:

Yes, thank you. I would like to know:

- What cloud technologies and DevOps tools does your team primarily use?
- What are the biggest challenges the team is currently working on?
- Are there opportunities to work on Kubernetes, Platform Engineering, or AI-driven DevOps initiatives?

